

How YelpCamp Uses Cloudinary

Media upload, storage, rendering, and deletion — a walkthrough of the full image lifecycle • July 2026

1. Overview

YelpCamp never stores image files on its own server or in its database. Instead, it uses **Cloudinary** as a dedicated media platform: every photo a user attaches to a campground is streamed directly to Cloudinary, which hosts the binary file and serves it back over its global CDN. What YelpCamp keeps in **MongoDB** is only a lightweight *reference* to each image — two strings: the image's public delivery `url` and its Cloudinary `filename` (the "public ID"). The URL is used to display the image in the browser; the filename is kept so the asset can be deleted from Cloudinary later.

This separation of responsibilities is the core idea to understand: **Cloudinary owns the pixels, MongoDB owns the pointers**. The Express application sits in the middle and coordinates the two — it forwards uploads to Cloudinary, saves the returned references into Mongo, embeds the URLs into rendered pages, and, on deletion, removes the asset from Cloudinary first and then pulls the reference out of Mongo.

2. Configuration

All Cloudinary wiring lives in a single module, `cloudinary/index.js`. It does two things. First, it configures the official Cloudinary SDK using three environment variables (kept out of source control in `.env` and loaded by `dotenv` at the top of `app.js`): `CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_KEY`, and `CLOUDINARY_SECRET`. These credentials identify the account and authorize write operations such as uploads and deletions.

Second, it creates a `CloudinaryStorage` instance from the `multer-storage-cloudinary` package. This is a *storage engine for multer* (the Express middleware that parses file uploads). It tells multer: instead of writing incoming files to local disk, stream them straight to Cloudinary, place them inside a folder named `YelpCamp`, and only accept `jpeg`, `png`, and `jpg` formats.

`cloudinary/index.js`

```
const cloudinary = require('cloudinary').v2;
const { CloudinaryStorage } = require('multer-storage-cloudinary');

cloudinary.config({
  cloud_name: process.env.CLOUDINARY_CLOUD_NAME,
  api_key: process.env.CLOUDINARY_KEY,
  api_secret: process.env.CLOUDINARY_SECRET
});

const storage = new CloudinaryStorage({
  cloudinary,
  params: {
    folder: 'YelpCamp',
    allowedFormats: ['jpeg', 'png', 'jpg']
  }
});

module.exports = { cloudinary, storage }
```

The module exports both objects: `storage` is consumed by the routes (for uploads), and the configured `cloudinary` instance is consumed by the campgrounds controller (for deletions).

3. Uploading images

The upload journey starts in the browser. Both the "new campground" form (`views/campgrounds/new.ejs`) and the edit form declare `enctype="multipart/form-data"` — without this attribute the browser would not transmit file contents at all. The file picker is named `image` and carries the `multiple` attribute, so a user can attach several photos in one submission:

`views/campgrounds/new.ejs`

```
<form action="/campgrounds" method="POST" novalidate class="validated-form" enctype="multipart/form-data">
  ...
  <input class="form-control" type="file" id="image" name="image" multiple>
```

On the server, `routes/campgrounds.js` constructs a multer instance backed by the Cloudinary storage engine and mounts it as route middleware:

`routes/campgrounds.js`

```
const multer = require('multer');
const { storage } = require('../cloudinary');
const upload = multer({ storage });

router.route('/')
  .get(catchAsync(campgrounds.index))
  .post(isLoggedIn, upload.array('image'), validateCampground, catchAsync(campgrounds.createCampground));
```

The middleware chain order matters and is worth being able to narrate:

1. `isLoggedIn` — rejects unauthenticated users before any file is processed.
2. `upload.array('image')` — multer parses the multipart body. Because its storage engine is `CloudinaryStorage`, each file is *streamed to Cloudinary at this moment*, before the controller ever runs. Cloudinary stores the binary in the `YelpCamp` folder and responds with the asset's details. Multer then populates `req.files` (an array, one entry per file) and `req.body` (the text fields).
3. `validateCampground` — Joi schema validation of the now-parsed text fields.
4. `createCampground` — the controller, which finally persists the campground.

Each entry in `req.files` carries two properties the app cares about: `path`, which is the full HTTPS delivery URL of the freshly uploaded asset, and `filename`, which is the Cloudinary public ID (including the folder prefix, e.g.

`YelpCamp/ab12cd34`). The controller maps those onto the campground document:

`controllers/campgrounds.js - createCampground`

```
const campground = new Campground(req.body.campground);
campground.geometry = geoData.body.features[0].geometry;
campground.images = req.files.map(f => ({ url: f.path, filename: f.filename }));
campground.author = req.user._id;
await campground.save();
```

So by the time `campground.save()` runs, the images already live on Cloudinary; Mongo receives only the URL/filename pairs.

4. Storing image references in MongoDB

The Campground model (`models/campground.js`) defines a small embedded subdocument schema for images and stores them as an array on each campground:

`models/campground.js`

```
const ImageSchema = new Schema({
  url: String,
  filename: String
});

ImageSchema.virtual('thumbnail').get(function () {
  return this.url.replace('/upload', '/upload/w_200');
});

const opts = { toJSON: { virtuals: true } };

const CampgroundSchema = new Schema({
  title: String,
  images: [ImageSchema],
  ...
}, opts);
```

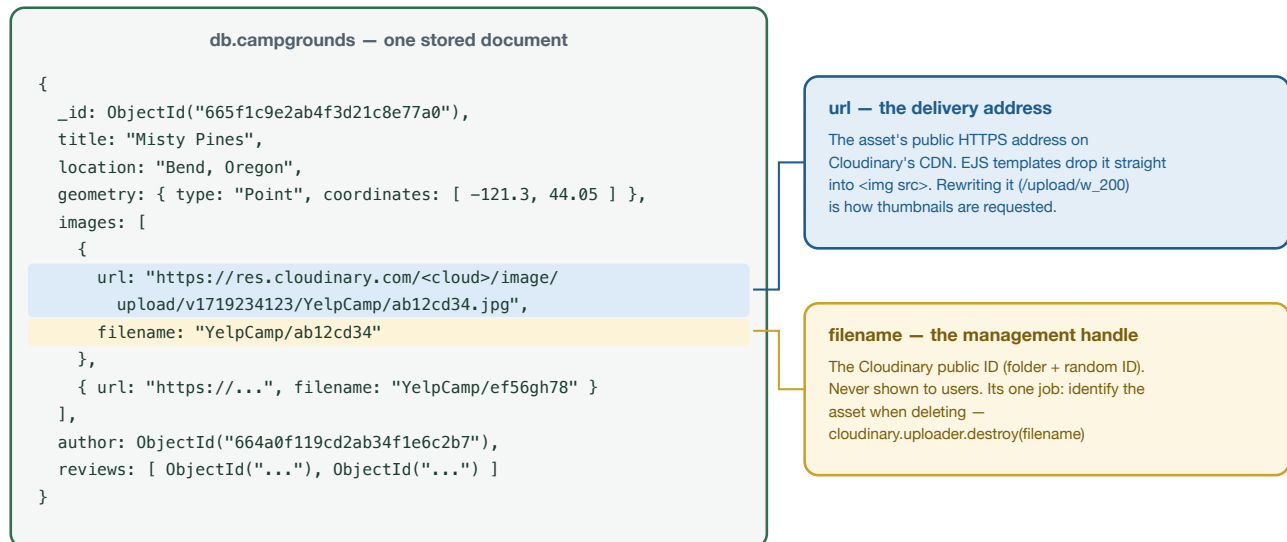
Each of the two stored fields has one job:

Field	Example value	Purpose
<code>url</code>	<code>https://res.cloudinary.com/<cloud>/image/upload/v.../YelpCamp/ab12cd34.jpg</code>	Delivery. Dropped into <code></code> tags so the browser fetches the image from Cloudinary's CDN.
<code>filename</code>	<code>YelpCamp/ab12cd34</code>	Management. This is the Cloudinary public ID — the handle required to delete (or otherwise manipulate) the asset via the API later.

Because images are embedded subdocuments rather than a separate collection, a campground and its image references are always loaded together in one query, and array operators like `$pull` can remove references atomically. The `toJSON: { virtuals: true }` option matters too: it ensures computed virtual properties (like `thumbnail`, section 5) survive `JSON.stringify`, which the show page relies on when it serializes the campground for the client-side map script.

4.1 Illustration — what MongoDB actually stores

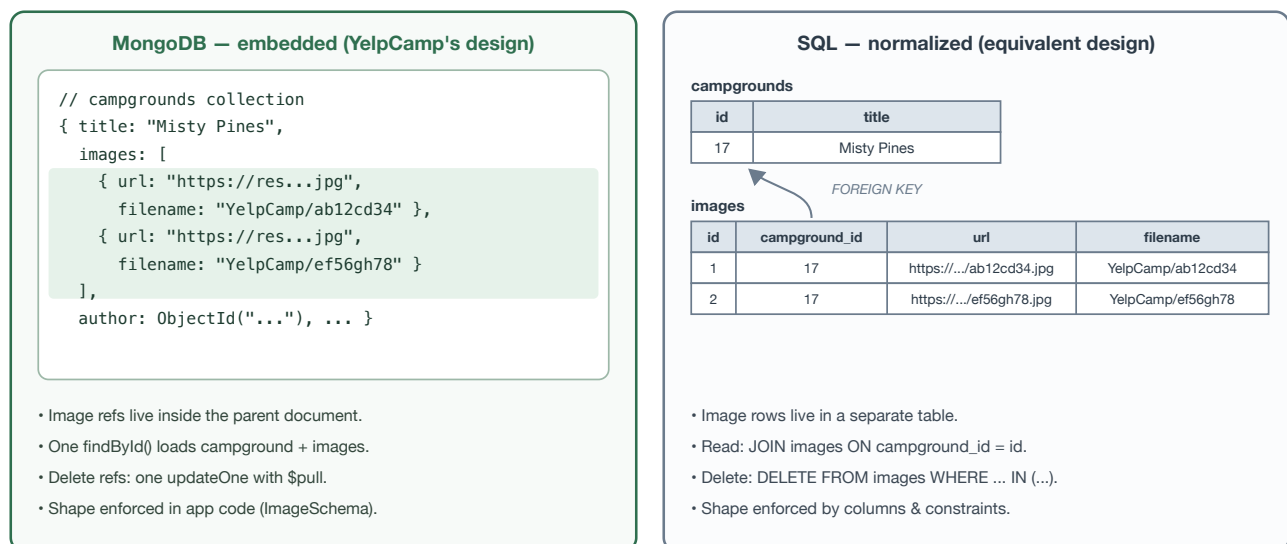
Concretely, here is what a single campground document looks like inside the `campgrounds` collection after two photos have been uploaded. Notice that the `images` array contains nothing but short strings — roughly two hundred bytes per photo — no matter how large the original files were. The two highlighted fields are the two halves of the Cloudinary contract:



The image binaries are not in this document — MongoDB holds only reference strings, while the pixels live in the Cloudinary `YelpCamp` folder.

4.2 Illustration — this design vs. a SQL database

YelpCamp uses a *document* model: because each image belongs to exactly one campground, is only ever displayed alongside it, and the list is small and bounded, the references are **embedded** directly inside the campground document. A relational (SQL) database would model the same one-to-many relationship by **normalizing**: the image references would move into their own `images` table, tied back to the campground by a foreign key. Both designs store exactly the same two strings per photo — the difference is purely *where the references live* and how you read and modify them:



Same data, two shapes: MongoDB nests the image references inside the campground; SQL splits them into a child table joined by a foreign key.

Operation by operation, here is how the two designs compare for this app's image handling:

Operation	MongoDB — embedded (YelpCamp)	SQL — normalized
Load a campground with its photos	One query: <code>Campground.findById(id)</code> — images arrive embedded in the document.	A <code>JOIN</code> onto the <code>images</code> table (or a second query per campground).
Add photos after upload	<code>campground.images.push(...imgs)</code> then <code>save()</code> .	<code>INSERT INTO images (campground_id, url, filename) VALUES ...</code> one row per photo.
Remove selected photos	One update: <code>\$pull</code> matching filenames out of the array.	<code>DELETE FROM images WHERE campground_id = ? AND filename IN (...)</code> .
Data shape / integrity	Defined in application code by Mongoose's <code>ImageSchema</code> ; the database itself is schemaless.	Enforced by the database: column types, <code>NOT NULL</code> , and the foreign-key constraint.
The image binary itself	Identical in both designs: never in the database. Only the two reference strings are stored; the file lives on Cloudinary. (Databases <i>can</i> hold binaries — BLOB columns, GridFS — but that bloats them and forfeits Cloudinary's CDN and transformations.)	

Embedding is the right call here because the images are a small, bounded list read exclusively through their parent campground. The trade-off to acknowledge if asked: embedded data is harder to query *across* parents (e.g. "list every image in the system" means unwinding arrays, where SQL would just scan the `images` table), and a document has a 16 MB cap — irrelevant at a few hundred bytes per image reference.

5. Displaying images

Display is deliberately simple: the app never proxies image bytes. EJS templates emit `<img>` tags whose `src` points at Cloudinary, and the user's browser downloads each image *directly from Cloudinary's CDN* — the Express server is not involved in serving pixels.

Full-size images (show page carousel)

`views/campgrounds/show.ejs` loops over `campground.images` inside a Bootstrap carousel and uses the stored URL as-is; the previous/next controls render only when there is more than one image:

`views/campgrounds/show.ejs`

```
<% campground.images.forEach((img, i) => { %>
  <div class="carousel-item <%= i === 0 ? 'active' : '' %>">
    
  </div>
<% }) %>
```

On-the-fly thumbnails via URL transformation (edit page)

Cloudinary supports *URL-based transformations*: modifying the delivery URL itself instructs Cloudinary to derive a new version of the image on the fly (and cache it). YelpCamp uses exactly one such transformation, defined as the `thumbnail` virtual on `ImageSchema`:

```
ImageSchema.virtual('thumbnail').get(function () {
  return this.url.replace('/upload', '/upload/w_200');
});
```

Inserting the `w_200` segment after `/upload` asks Cloudinary for a 200-pixel-wide rendition. Nothing extra is stored — it is a computed property derived from `url` whenever it is read. The edit page uses it so users can review existing photos as small thumbnails while deciding which to delete:

views/campgrounds/edit.ejs

```

```

Index page with fallback

The campground list (`views/campgrounds/index.ejs`) shows each campground's first image, `campground.images[0].url`. If a campground has no images at all, it falls back to a hardcoded default image URL that also lives in the Cloudinary `YelpCamp` folder, so every card always has a picture.

6. Deleting images

Image deletion happens through the **edit form**. Next to each existing thumbnail, the template renders a "Delete?" checkbox whose `name` is `deleteImages[]` and whose `value` is that image's `filename` — the Cloudinary public ID:

views/campgrounds/edit.ejs

```
<% campground.images.forEach(function(img, i) { %>
  
  <div class="form-check-inline">
    <input type="checkbox" id="image-<%= i %>" name="deleteImages[]" value="<%= img.filename %>">
  </div>
  <label for="image-<%= i %>">Delete?</label>
<% }) %>
```

When the form is submitted (a PUT to `/campgrounds/:id`), the checked values arrive as the array `req.body.deleteImages`. The `updateCampground` controller then performs a **two-step, two-system deletion**:

controllers/campgrounds.js — `updateCampground`

```
if (req.body.deleteImages) {
  for (let filename of req.body.deleteImages) {
    await cloudinary.uploader.destroy(filename);
  }
  await campground.updateOne({ $pull: { images: { filename: { $in: req.body.deleteImages } } } });
}
```

1. **Cloudinary first:** `cloudinary.uploader.destroy(filename)` is called once per selected image. This is an authenticated API call (using the credentials from section 2) that permanently removes the binary asset from the Cloudinary account. This is exactly why `filename` was stored at upload time — it is the only handle Cloudinary accepts for deletion.
2. **Then MongoDB:** a single `updateOne` with the `$pull` operator removes every embedded image subdocument whose `filename` is in the submitted list, keeping the database in sync so dead URLs are never rendered.

7. The combined edit flow

The update route composes everything above into one request. Its middleware chain mirrors the create route, with an added ownership check:

routes/campgrounds.js

```
router.route('/:id')
  .put(isLoggedIn, isAuthor, upload.array('image'), validateCampground,
    catchAsync(campgrounds.updateCampground))
```

A single PUT submission can therefore simultaneously:

- **Add images** — new files chosen in the edit form's file input are uploaded to Cloudinary by `upload.array('image')`, and the controller pushes the resulting `{ url, filename }` pairs onto the existing array:
`campground.images.push(...imgs)`.
- **Remove images** — any checked `deleteImages[]` entries are destroyed on Cloudinary and `$pull`-ed from Mongo, as described in section 6.
- **Update text fields** — via `Campground.findByIdAndUpdate(id, { ...req.body.campground })`.

8. Limitations and honest caveats

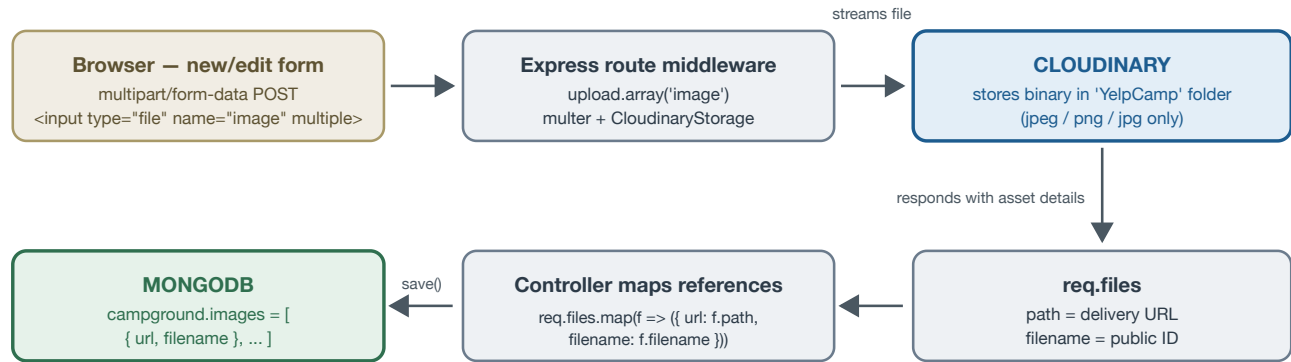
Orphaned assets on campground deletion. Deleting an entire campground calls `Campground.findByIdAndDelete(id)`, and a `findOneAndDelete` post-hook on the model cascades deletion of the campground's reviews — but nothing in that path calls Cloudinary. The campground's images therefore remain in the Cloudinary account as orphaned assets. The clean fix would be to extend the post-hook (or the `deleteCampground` controller) to loop over `doc.images` and call `cloudinary.uploader.destroy(img.filename)` for each.

Upload happens before validation. Because `upload.array('image')` must run before `validateCampground` (multer is what parses the multipart body that validation needs to read), files are already on Cloudinary by the time validation runs. If validation then rejects the request, the uploaded files are not cleaned up. Format restrictions (`jpeg/png/jpg`) are enforced by the storage engine itself, but there is no file-count or file-size cap configured on multer.

9. Diagram: the media lifecycle

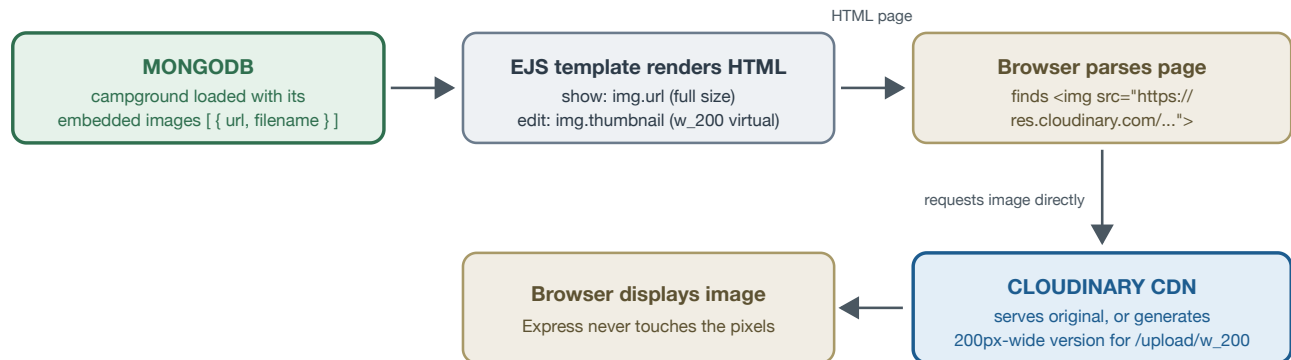
Three flows, one shared principle: **Cloudinary stores the binary media**; **MongoDB stores only the `{ url, filename }` references**; the Express app coordinates both, and the browser fetches pixels straight from Cloudinary's CDN.

Flow 1 — Upload & Store



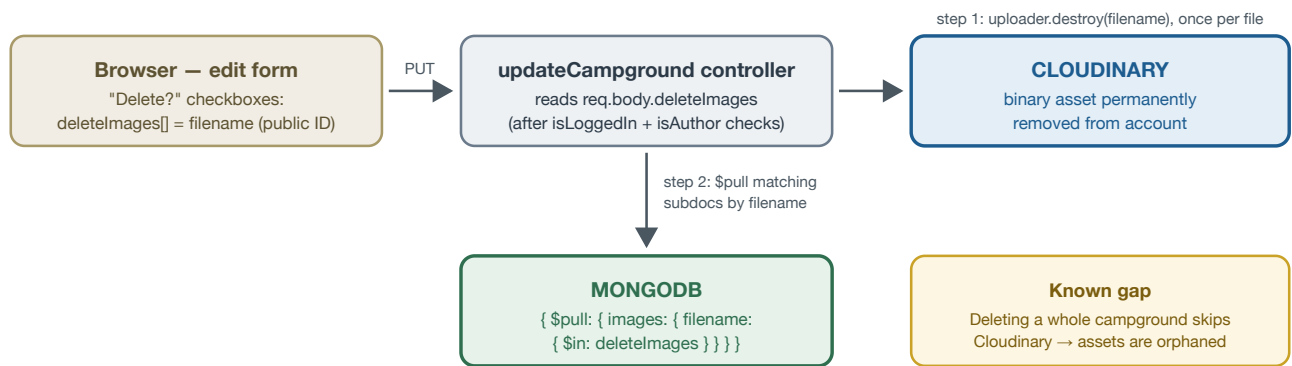
Files are uploaded to Cloudinary by the multer middleware before the controller runs; MongoDB receives only the returned URL and public ID.

Flow 2 — Render / Display



The Express server only embeds URLs into HTML; image bytes travel directly from Cloudinary's CDN to the user's browser.

Flow 3 — Delete



Deletion is two-phase: destroy the asset on Cloudinary using its stored public ID, then pull the reference out of the MongoDB array.